



BAIT AL MANDI

System Architecture Documentation

RESTAURANT ORDER MANAGEMENT SYSTEM

Version	1.0
Document ID	BAM-ARCH-EN-20260616
Generated	June 16, 2026
Document Type	System Architecture Document

DEVELOPED BY



Esnaad Tech

Technology Solutions & Software Development

Development Credits & Technical Ownership

This system was developed by:



Esnaad Tech

Technology Solutions & Software Development

All technical and intellectual property rights reserved.

This document is the proprietary property of Esnaad Tech.

© 2026 Esnaad Tech. All rights reserved.

Table of Contents

Bait Al Mandi - Full System Architecture Document...1

Phase 1: Project Index2

Phase 2: System Overview3

Phase 3: Page Analysis4

Phase 4: Database Analysis5

Phase 5: Order Lifecycle6

Phase 6: Offer and Bundle System7

Phase 7: Invoice System8

Phase 8: Tracking Token System9

Phase 9: Store Analysis10

Phase 10: Server Actions Analysis11

Phase 11: Security Analysis12

Phase 12: Performance Analysis13

Phase 13: Architecture Analysis14

Phase 14: Dependency Map15

Phase 15: Final Report16

Architecture Summary

Overview of technologies and services used in the system

 Architecture Hybrid: Layered + Feature-Based	 Framework Next.js 14 (App Router)	 Language TypeScript
 Styling TailwindCSS + CSS Variables	 Database Supabase (PostgreSQL) + Drizzle ORM	 Auth Supabase Auth + RLS
 State Zustand + localStorage	 Maps Leaflet + OSRM	 Invoices jsPDF + html2canvas
 Analytics PostHog	 Monitoring Sentry	 Audit audit_logs

Database Schema Summary

System tables and their relationships — 16 tables total

Table	Description	Key Fields	Relations
users	المستخدمون	id, full_name, phone	orders (1:N)
admin_users	المشرفون	id, email, role, auth_user_id	order_status_history (1:N)
categories	التصنيفات	id, name_ar, name_en, slug	items (1:N)
items	الأطباق	id, category_id (FK), name_ar	item_prices (1:N), offer_items
item_prices	الأسعار	id, item_id (FK), sale_price	items (N:1)
offers	العروض	id, title_ar, offer_type	offer_items (1:N), orders
offer_items	مكونات العرض	id, offer_id (FK), menu_item_id (FK)	offers, items, item_prices
orders	الطلبات	id, order_number, status, total	users, order_items (1:N)
order_items	عناصر الطلب	id, order_id (FK), item_name	orders (N:1)
order_status_history	سجل الحالة	id, order_id (FK), new_status	orders, admin_users
gallery_images	المعرض	id, image_url, show_on_homepage	—
reviews	التقييمات	id, reviewer_name, rating, comment	—
site_settings	الإعدادات	id, setting_key (JSONB)	—
branches	الفروع	id, name_ar, address, phone	—
audit_logs	التدقيق	id, action, details (JSON)	admin_users
customer_tokens	الرموز	phone (PK), tracking_token	orders (1:1)

System Flows

Order creation flow from browsing to delivery

1

Browse

Visit /menu, view items/offers, add to cart

2

Cart

Fill data, select location, calculate delivery fee

3

Order

createOrder() → Validation → Calc → DB Insert (4-6 tables)

4

Invoice

InvoiceModal → QR Code → PDF

5

Tracking

/t/[token] ← Secure link with order status

6

Management

admin/orders → Status update with optimistic locking

7

Reports

Daily/weekly/monthly analytics with export

Core Technologies

Distance

Haversine + OSRM

Delivery Fee

base_fee + extra_km + weather + peak

Order #

BAM-YYYYMMDD-XXXX

Tracking

UUID v4 (persistent per customer)

Offers

calculateOfferPrice() SSoT

Concurrency

Optimistic Locking (version)

Bait Al Mandi - Full System Architecture Documentation

Restaurant Order Management System - Complete Technical Reference

Phase 1: Project Index

Project Tree

```
baitalmandiwibapp/

├─ package.json           # Project definition and dependencies
├─ next.config.js         # Next.js configuration (images, optimization)
├─ tailwind.config.ts     # TailwindCSS configuration
├─ tsconfig.json          # TypeScript configuration
├─ .env.local             # Environment variables (Supabase, Sentry, etc.)
├─ drizzle.config.ts      # Drizzle ORM configuration
├
├─ supabase/
│   └─ migrations/        # Migration files (12 files)
│       └─ 0000_small_ozymandias.sql  # Core structure: all tables
│       └─ 0001_remarkable_terrax.sql  # Pricing modifications
│       └─ 0002_fix_rls_policies.sql   # RLS policy fixes
│       └─ 0003_add_tracking_token.sql  # Add tracking_token to orders
│       └─ 0004_add_offer_bundles.sql   # Add offer_items for bundles
│       └─ 0005_add_customer_tokens.sql # Add customer_tokens table
│       └─ 0006_add_order_offers.sql    # Add Snapshot tables for offers
│       └─ 0007_add_variant_to_offers.sql # Variant support in offer_items
│       └─ 0008_add_delivery_system.sql  # New delivery system
│       └─ 0009_add_new_delivery_pricing.sql # Enhanced delivery pricing
│       └─ 0010_add_delivery_snapshot.sql  # Delivery data snapshot
```



```
|   └─ 0011_add_show_on_homepage.sql    # Add show_on_homepage field
|
|   └─ meta/                            # Migration snapshot data
|
|
└─ src/
  |
  |   └─ middleware.ts                  # Middleware for admin route protection
  |
  |   └─ types.d.ts                    # Global type declarations
  |
  |   |
  |   └─ app/                          # Next.js 14 App Router pages
  |     |
  |     |   └─ layout.tsx              # Root layout (Navbar, Footer, Theme)
  |     |
  |     |   └─ page.tsx                # Home page (Hero, Offers, Gallery, Reviews)
  |     |
  |     |   └─ globals.css             # Global styles and CSS variables
  |     |
  |     |   |
  |     |   └─ menu/
  |     |     |
  |     |     |   └─ page.tsx          # Menu page (items + offers)
  |     |     |
  |     |     |
  |     |     └─ gallery/
  |     |       |
  |     |       |   └─ page.tsx        # Public gallery
  |     |       |
  |     |       |
  |     |       └─ contact/
  |     |         |
  |     |         |   └─ page.tsx      # Contact page
  |     |         |
  |     |         |
  |     |         └─ cart/
  |     |           |
  |     |           |   └─ page.tsx    # Cart page
  |     |           |
  |     |           |
  |     |           └─ my-orders/
```

```

| | | └─ page.tsx      # Order creation page

| | |

| | └─ t/

| | | └─ [token]/

| | |     └─ page.tsx    # Token-based order tracking

| | |

| | └─ track-order/

| | | └─ [orderId]/

| | |     └─ page.tsx    # ID-based order tracking

| | |

| | └─ test-map/

| | | └─ page.tsx      # Map testing page

| | |

| | └─ admin/          # Admin dashboard

| |     └─ layout.tsx  # Admin layout (Sidebar, Auth)

| |     └─ page.tsx    # Admin dashboard home

| |     └─ login/

| |         └─ page.tsx # Login page

| |         └─ orders/

| |             └─ page.tsx # Order management

| |             └─ actions.ts # Server Actions for orders

| |             └─ menu/

| |                 └─ page.tsx # Menu item management

| |                 └─ categories/

| |                     └─ page.tsx # Category management

```

```
| |   └─ offers/
| |   |   └─ page.tsx      # Offer/bundle management
| |   └─ gallery/
| |   |   └─ page.tsx      # Gallery management
| |   └─ reviews/
| |   |   └─ page.tsx      # Review management
| |   └─ reports/
| |   |   └─ page.tsx      # Reports and statistics
| |   └─ delivery/
| |   |   └─ page.tsx      # Delivery settings
| |   └─ settings/
| |       └─ page.tsx      # Site settings
| |
| └─ actions/              # Server Actions
| |   └─ orders.ts         # createOrder, getOrders, updateOrderStatus
| |   └─ orders-offers.ts  # getOrderOffers
| |
| └─ components/           # Shared components
| |   └─ layout/
| |       └─ Navbar.tsx    # Navigation bar
| |       └─ Footer.tsx    # Footer
| |       └─ LoadingScreen.tsx # Loading screen
| |   └─ ui/
| |       └─ Toast.tsx      # Toast notification system
| |   └─ invoice/
```

```
| | | └─ InvoiceModal.tsx # Invoice modal dialog
| | | └─ receipt-html.ts # HTML receipt generation
| | └─ admin/
| |     └─ reports/
| |         └─ OffersTab.tsx # Offer reports
| |         └─ InvoicesTab.tsx # Invoice reports
| |
| └─ store/ # State management (Zustand)
|     └─ cartStore.ts # Cart store with local persistence
|
| └─ lib/ # Services and utilities
|     └─ supabase.ts # Supabase client (public)
|     └─ offer-pricing.ts # Offer pricing engine (SSoT)
|     └─ bundle-utils.ts # Bundle utilities and backward compatibility
|     └─ delivery-pricing.ts # Delivery fee engine (SSoT)
|     └─ delivery-routing.ts # Routing and delivery services
|     └─ location-validation.ts # Location validation
|     └─ whatsapp-message.ts # WhatsApp message builder
|     └─ homepage-gallery.ts # Homepage gallery service
|     └─ settings-context.tsx # Site settings context
|     └─ permissions.ts # Permission system
|     └─ ordering.ts # Sort ordering utilities
|     └─ printReport.ts # Report printing
|     └─ validation.ts # Data validation
|     └─ maps/
```

```
| |      └─ getDeliveryRoute.ts # OSRM routing  
  
| |  
  
| └─ db/                                # Database  
|   └─ schema.ts                      # Drizzle ORM table definitions  
|     └─ rls.sql                      # Row Level Security policies  
  
| |  
  
| └─ utils/  
|   └─ supabase/                      # Supabase utilities  
|     └─ client.ts                   # Browser client  
|     └─ server.ts                   # Server client  
|     └─ middleware.ts               # Middleware client
```

Phase 2: System Overview

What is this system?

An **integrated restaurant order management system** for "Bait Al Mandi" restaurant in Sanaa, Yemen. It allows customers to browse the menu, view offers, add items to cart, and create delivery orders with real-time tracking.

What problems does it solve?

1. **Order Automation:** Transforms manual phone orders into a structured electronic system

- 2. **Delivery Management:** Automatically calculates delivery fees based on distance and conditions
- 3. **Order Tracking:** Enables customers to track their orders in real-time
- 4. **Restaurant Management:** Provides a comprehensive admin dashboard for managers and staff
- 5. **Reporting:** Generates integrated financial and administrative reports

User Types

User Type	Description	Permissions
---	---	---
Developer	Full technical administrator	All permissions without exception
Manager	Restaurant manager	Orders, items, categories, offers, gallery, reviews, reports, settings
Order Manager	Order reception staff	Orders only (view, update status)
Customer	Website visitor	Browse menu, create orders, track orders

Phase 3: Page Analysis

Route	Page	Purpose	Access	Data	Services
---	---	---	---	---	---
/	Home	Display offers, story, reviews	Public	offers, reviews, gallery_images, site_settings	supabase, offer-pricing, homepage-gallery
/menu	Menu	Display items and offers with add-to-cart	Public	categories, items, item_prices, offers, offer_items	supabase, offer-pricing, cartStore
/gallery	Gallery	Display restaurant images with filter/lightbox	Public	gallery_images	supabase
/contact	Contact	Branch information and communication	Public	site_settings, branches	supabase
/cart	Cart	View and modify cart contents	Public	cartStore (local)	cartStore
/my-orders	Order creation	Order form with location selection	Public	orders, site_settings, offers	supabase, orders.ts, cartStore
/t/[token]	Order tracking	Secure token-based order tracking	Public	orders, order_items, order_offers	supabase
/track-order/[orderId]	Order tracking	ID-based order tracking	Public	orders, order_offers	supabase, orders-offers

Route	Page	Purpose	Access	Data	Services
<code>/admin</code>	Dashboard	Summary and quick statistics	Admin	orders	supabase
<code>/admin/orders</code>	Orders	Manage and display orders, update status	Admin	orders, order_items	supabase, orders.ts
<code>/admin/menu</code>	Menu items	Add, edit, delete menu items	Admin	items, item_prices, categories	supabase
<code>/admin/categories</code>	Categories	Manage menu categories	Admin	categories	supabase
<code>/admin/offers</code>	Offers	Create and manage offers/bundles	Admin	offers, offer_items, items, item_prices	supabase, offer-pricing
<code>/admin/gallery</code>	Gallery	Manage restaurant images	Admin	gallery_images	supabase
<code>/admin/reviews</code>	Reviews	Manage customer reviews	Admin	reviews	supabase
<code>/admin/reports</code>	Reports	Advanced sales analytics	Admin	orders, order_offers	supabase
<code>/admin/delivery</code>	Delivery	Delivery configuration and zones	Admin	site_settings, branches	supabase
<code>/admin/settings</code>	Settings	General site settings	Admin	site_settings	supabase

Phase 4: Database Analysis

Logical ERD

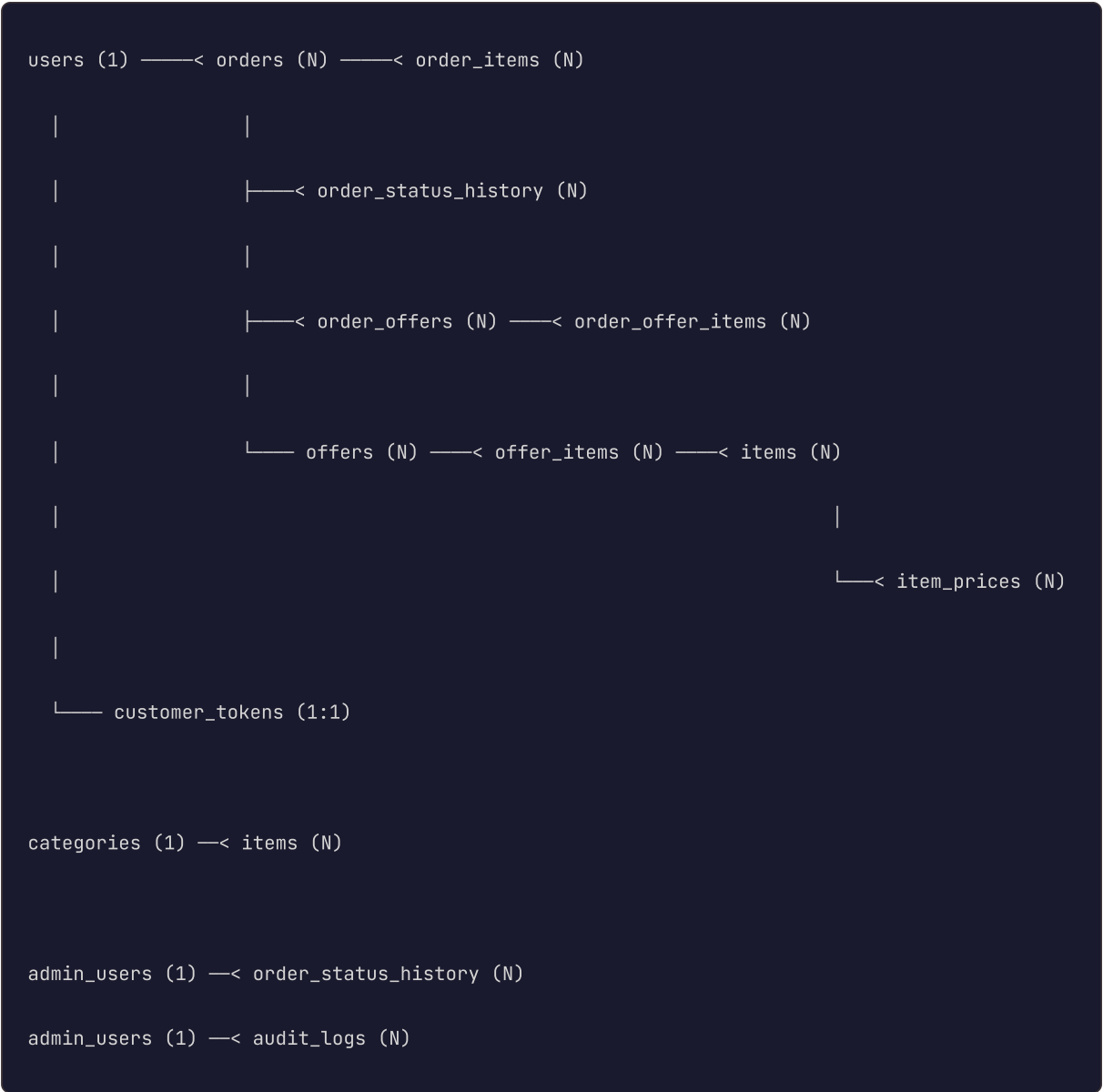


Table Details

1. users — Customers

- **Purpose:** Store registered customer data

- **Fields:** id, full_name, phone (unique), address, created_at

- **Relations:** Linked to orders via customer_id

2. admin_users — Admin Users

- **Purpose:** Store admin and staff data

- **Fields:** id, email (unique), full_name, role (enum), auth_user_id

- **Roles:** developer, manager, order_manager

- **Note:** Linked to Supabase Auth via auth_user_id

3. categories — Item Categories

- **Purpose:** Categorize menu items (e.g., Mandi, Zurbian, Drinks)

- **Fields:** id, name_ar, name_en, slug (unique), icon, image, sort_order, is_active

- **Relations:** Linked to items via category_id

4. items — Menu Items

- **Purpose:** Store dish/menu item data

- **Fields:** id, category_id (FK), name_ar, name_en, description_ar, description_en, image, is_best_seller, is_available, is_active, sort_order

- **Relations:**

- belongs_to: categories (category_id)

- has_many: item_prices (different sizes/prices)

- has_many: offer_items (as bundle components)

5. item_prices — Item Prices by Size

- **Purpose:** Store different prices for the same item (small/medium/large)
- **Fields:** id, item_id (FK), size_label_ar, size_label_en, serves, original_price, sale_price, is_active
- **Relations:** belongs_to: items

6. offers — Offers and Bundles

- **Purpose:** Store promotional offers and bundled packages
- **Fields:** id, item_id (legacy FK), title_ar, title_en, description_ar, description_en, discount_percent, sale_price, discount_amount, offer_type, is_active, start_date, end_date, image, status, deleted_at
- **Offer Types:** fixed_price, percentage_discount, amount_discount, free_item
- **Relations:**
 - belongs_to: items (legacy - for single-item offers)
 - has_many: offer_items (for bundles)

7. offer_items — Bundle Components

- **Purpose:** Store products that make up a bundle with quantities and prices
- **Fields:** id, offer_id (FK), menu_item_id (FK), quantity, price_id (FK), variant_name, unit_price
- **Relations:**
 - belongs_to: offers

- belongs_to: items (via menu_item_id)
- belongs_to: item_prices (via price_id for size selection)

8-22. Remaining Tables

(See Arabic document for complete details on: cart_sessions, cart_items, orders, order_items, order_status_history, gallery_images, reviews, site_settings, branches, audit_logs, customer_tokens, order_offers, order_offer_items, scheduled_reports, order_sequences)

Phase 5: Order Lifecycle

Complete Order Flow

1. Browse Menu

- |
- |— Customer visits /menu
- |— Views items and offers
- |— Adds items to cart (cartStore)

2. Cart

- |
- |— Cart displayed in /my-orders
- |— Customer fills data (name, phone, address)
- |— Selects location on map (optional)
- |— Delivery fee shown automatically
- |— Selects payment method (cash/transfer/wallet)

3. Order Creation

- |
- |— Customer clicks "Complete Order"
- |— request → createOrder() Server Action
- |
- |— Validation:
 - |— Customer name (required)

```
|   |— Phone number (valid Yemeni format)

|   |— Address (minimum 10 characters)

|   |— Items (at least one required)

|   |— Distance (within delivery range)

|

|— Delivery Fee Calculation:

|   |— Haversine for straight-line distance

|   |— OSRM for actual road distance

|   |— base_fee + extra_km_fee

|   |— weather_fee (if applicable)

|   |— peak_hours_fee (if applicable)

|   |— Validate against max_delivery_distance_km

|

|— Order Number Generation:

|   |— Format: BAM-YYYYMMDD-XXXX

|   |— Uses order_sequences with optimistic locking

|   |— Random fallback if generation fails

|

|— Tracking Token Management:

|   |— Search for existing tracking_token by phone

|   |— Create new tracking_token if not found

|

|— Offer Calculation (if applicable):

|   |— Fetch offer with offer_items

|   |— Validate offer availability
```

```
|   |— Calculate price via calculateOfferPrice()

|   |— Add discounted price to order total

|

|— Minimum Order Validation:

|   |— total ≥ min_order_amount

|   |— Reject order if below minimum

|

|— Database Insertion:

|   |— Insert into orders

|   |— Insert into order_items

|   |— Insert into order_status_history

|   |— Insert into order_offers + order_offer_items (if offer)

|

|— Payment Processing:

|   |— cash: No additional action

|   |— transfer: Open WhatsApp with transfer message

|   |— wallet: Open WhatsApp with wallet message

|

|— response → order + tracking_token
```

4. Invoice

```
|

|— InvoiceModal: Display PDF invoice

|— receipt-html: Generate HTML invoice

|— QR Code: Tracking QR code
```

- └─ html2canvas + jsPDF: Convert HTML to PDF

5. Tracking

- |
- └─ /t/[token] ← Secure tracking link
- └─ Displays order status and location
- └─ Displays invoice details
- └─ Displays delivery map (optional)

6. Order Management (Admin)

- |
- └─ admin/orders: View all orders
- └─ Status update: pending → confirmed → preparing → on_the_way → delivered
- └─ Optimistic locking via version field
- └─ Every change logged in order_status_history
- └─ Order cancellation with audit trail

7. Reports

- |
- └─ Daily/weekly/monthly reports
- └─ Total sales
- └─ Offer statistics
- └─ Delivery analytics
- └─ Excel export

Phase 6: Offer and Bundle System

How the Offer System Works

The offer system is built on the **SSoT (Single Source of Truth)** principle where `calculateOfferPrice()` in `src/lib/offer-pricing.ts` is the sole source for offer price calculation across the entire system.

Offer Types

Type	Description	Example
---	---	---
<code>fixed_price</code>	Fixed price for the bundle regardless of components	Bundle at 1500 YER
<code>percentage_discount</code>	Percentage discount from total bundle value	20% off
<code>amount_discount</code>	Fixed amount discount from total	500 YER off
<code>free_item</code>	Cheapest item in bundle is free	Buy 3, pay for 2

Bundle Storage

When creating/editing an offer (via `/admin/offers`):

1. **Product Selection:** Admin selects bundle products with quantities
2. **Size Selection:** Selects size/variant for each product (optional)
3. **unit_price Calculation:** Effective price calculated via `resolveItemPrice()` which prefers `sale_price` over `original_price`
4. **Price Storage:** `unit_price` stored in `offer_items` table as backup snapshot

5. **Final Price Calculation:** Final price calculated via `calculateOfferPrice()`

calculateOfferPrice Algorithm

```
function calculateOfferPrice(input):  
  
    originalPrice = sum(product.unitPrice * product.quantity)  
  
    switch input.offerType:  
  
        fixed_price:      finalPrice = salePrice  
  
        percentage_discount: finalPrice = originalPrice * (1 - discountPercent/100)  
  
        amount_discount:   finalPrice = max(0, originalPrice - discountAmount)  
  
        free_item:         finalPrice = originalPrice - cheapestItemPrice  
  
    discountAmount = originalPrice - finalPrice  
  
    discountPercent = round(discountAmount / originalPrice * 100)  
  
    return { originalPrice, finalPrice, discountAmount, discountPercent }
```

Is calculateOfferPrice the Single Source of Truth?

Yes. All parts of the system use the same function:

- `/menu/page.tsx` - Display offers in menu
- `/admin/offers/page.tsx` - Price preview when creating offers
- `src/actions/orders.ts` - Offer price calculation during order creation

- `src/app/page.tsx` - Offer display in carousel (fixed)
-

Phase 7: Invoice System

Responsible Files

- `src/components/invoice/InvoiceModal.tsx` - Invoice modal component
- `src/components/invoice/receipt-html.ts` - HTML invoice generation

Invoice Creation Process

1. `InvoiceModal` opens after order creation or from tracking page
2. Order data (order, items, offer info) passed to component
3. Invoice displayed in fully formatted HTML
4. QR code generated for tracking (using `qrcode.react`)
5. Invoice convertible to PDF via `html2canvas` + `jsPDF`
6. Direct printing supported

Invoice Components

- **Order Number:** BAM-YYYYMMDD-XXXX
- **Customer Data:** Name, phone, address
- **Items:** Item name, size, quantity, price

- **Offers:** Offer name, original price, discount, final price
 - **Totals:** subtotal, delivery_fee, total_amount
 - **Payment Method:** cash, transfer, wallet
 - **QR Code:** Secure tracking link
-

Phase 8: Tracking Token System

What is tracking_token?

A **secure unique identifier** (UUID v4) used for order tracking without exposing the order number or sensitive customer data.

How is it created?

1. When creating a new order, the system searches `customer_tokens` by customer phone number
2. If a previous token exists, it's reused (constant token per customer)
3. If not found, a new token is created via `crypto.randomUUID()`

Security Benefits

- **Privacy:** Token cannot be guessed (random UUID)
- **Persistence:** Same customer has same token always (even with multiple orders)
- **Secure Binding:** Token linked only to phone number, no other sensitive data

- **Secure Path:** `/t/[token]` requires correct token for access

Potential Risks

- If `tracking_token` leaks, anyone can track that customer's orders
- Proposed solution: Add additional verification layer (e.g., SMS code) for tracking page access

Phase 9: Store Analysis

cartStore (Zustand + Persist)

File: `src/store/cartStore.ts`

State:

```
interface CartState {  
  items: CartItem[] // Cart items array  
}
```

Actions:

Action	Description
---	---
<code>addToCart(item)</code>	Add item (increments quantity if exists)
<code>removeFromCart(id)</code>	Remove item completely
<code>updateQuantity(id, qty)</code>	Update quantity (removes if 0)
<code>clearCart()</code>	Clear entire cart
<code>getCartTotal()</code>	Calculate total price
<code>getTotalItems()</code>	Calculate total item count

Persistence: Cart saved to `localStorage` under key `bam-cart-storage` via `zustand/middleware/persist`

CartItem Structure:

```
interface CartItem {  
  
    id: string;  
  
    name: string;  
  
    price: number;  
  
    quantity: number;  
  
    size?: string;  
  
    category?: string;  
  
    image?: string;  
  
    isOffer?: boolean;  
  
    offerId?: string;  
  
    offerType?: string;  
  
    originalPrice?: number;  
  
    discountAmount?: number;  
  
    discountPercent?: number;  
  
    bundleItems?: BundleSubItem[];  
  
}
```

Phase 10: Server Actions Analysis

Server Actions Table

Name	Function	Inputs	Outputs	Affected Tables
---	---	---	---	---
createOrder	Create new order	CreateOrderInput	order object	orders, order_status_history, order_offers, order_offer_items, customer_token, order_sequences
getOrders	Fetch orders list	filters (status, is_archived)	order[]	orders, order_status_history
getOrderById	Fetch order by ID	orderId	order	orders, order_status_history
updateOrderStatus	Update order status	orderId, newStatus, adminId	order	orders, order_status_history
calculateDeliveryFeeServer	Calculate delivery fee	lat, lng	{ fee, distanceKm, ... }	site_settings, branches
getOrderOffers	Fetch order offers	orderId	OrderOfferSnapshot[]	order_offers, order_offer_items

Critical Actions

1. `createOrder` — Most critical. Contains complex logic:

- Validates all inputs
- Calculates distance and fees
- Generates order number (with optimistic locking)
- Manages tracking_token
- Calculates offer pricing
- Creates records in 4-6 different tables

2. `updateOrderStatus` — Order status control with:

- Optimistic locking (using version field)
 - Every change logged in order_status_history
-

Phase 11: Security Analysis

Authentication

- **Supabase Auth:** Uses Supabase for authentication
- **Admin Login:** `/admin/login` connects to Supabase Auth
- **Middleware:** `src/middleware.ts` protects all `/admin/*` routes
- **Session:** Verified via `@supabase/ssr`

Authorization

- **Role System:** developer, manager, order_manager
- **Client Verification:** `src/lib/permissions.ts` defines permissions per route
- **Server Verification:** Role verified in Server Actions

Tracking Protection

- **Tracking Token:** Random UUID, difficult to guess
- **Phone Binding:** Token linked to phone number only

Current Security Risks

Risk	Description	Severity
---	---	---
No Rate Limiting	Unlimited requests possible	Medium
tracking_token leak	Customer orders trackable if link leaks	Low
No CSRF protection	Server Actions potentially vulnerable to CSRF	Low
Supabase RLS only	Some queries depend solely on RLS	Medium
Database password in ENV	DB password in .env.local	High

Phase 12: Performance Analysis

Heavy Queries

1. **Orders page (/admin/orders)**: Fetches all orders with items in one query
2. **Reports page (/admin/reports)**: Complex analytics on Data Warehouse
3. **Order creation**: 5-7 sequential queries in createOrder

Potential N+1 Issues

1. **/menu page**: Items fetched with prices (good - uses nested select)
2. **/admin/offers page**: Offers fetched with products (nested select)
3. **createOrder**: Separate query for offer then items

Performance Recommendations (Priority Ordered)

1. **High**: Add Indexes on order_sequences.sequence_date and orders.created_at
 2. **High**: Add Pagination on /admin/orders
 3. **Medium**: Use React.Suspense with data preloading
 4. **Medium**: Optimize report queries with Materialized Views
 5. **Low**: Add Redis/Upstash caching for site settings
-

Phase 13: Architecture Analysis

Current Architecture

The system uses a **Hybrid Architecture** combining:

1. Layered Architecture:

- Presentation Layer: Components
- Business Logic Layer: Services and utilities (Lib)
- Data Layer: Database (DB)
- API Layer: Server Actions + API Routes

2. Feature-Based Organization:

- Each feature in separate files
- Pages organized by function

Strengths

- Uses Server Actions to minimize API Routes
- Implements SSoT for pricing and fees
- Snapshot for critical data (order_items, order_offers)
- Optimistic locking for order updates

Weaknesses

- No separate Service layer (logic in pages directly)
- Data fetching logic duplicated across pages
- No unified Validation Layer
- Mixing Client and Server logic in some files

Proposed Architecture

```
src/
├─ app/                # Pages (Presentation Layer)
├─ components/         # Shared Components
├─ features/           # Feature Modules
│   └─ orders/
│       ├── actions.ts  # Server Actions
│       ├── service.ts  # Business Logic
│       ├── types.ts    # Types
│       └─ components/  # Feature Components
│   └─ offers/
│       └─ delivery/
├─ lib/                # Core Services
├─ db/                 # Database
└─ store/              # State Management
```

Phase 14: Dependency Map

```
Home Page (/)

|

├─ supabase (gallery, offers, reviews)
|   └─ gallery_images, offers, reviews tables
|
|
├─ calculateOfferPrice (offer prices)
|   └─ offer-pricing.ts
|
|
├─ fetchHomepageGalleryImages (gallery)
|   └─ homepage-gallery.ts
|
|
├─ useSettings (site settings)
|   └─ settings-context.tsx
|
|
└─ cartStore (cart state)
    └─ cartStore.ts


Menu Page (/menu)

|

├─ supabase (items, categories, offers)
|   └─ items, categories, offers, offer_items tables
|
|
└─ calculateOfferPrice + resolveItemPrice
```

```
|   └─ offer-pricing.ts
```

```
|
```

```
└─ useSettings
```

```
|   └─ settings-context.tsx
```

```
|
```

```
└─ cartStore
```

```
    └─ cartStore.ts
```

```
Order Creation (createOrder)
```

```
|
```

```
└─ Server Actions
```

```
|   └─ orders.ts
```

```
|       └─ fetchDeliverySettings (site_settings + branches)
```

```
|       └─ calculateRoute (OSRM)
```

```
|       └─ calculateDeliveryFee (delivery-pricing.ts)
```

```
|       └─ calculateOfferPrice (offer-pricing.ts)
```

```
|       └─ generateOrderNumber (order_sequences)
```

```
|       └─ manageTrackingToken (customer_tokens)
```

```
|       └─ createOrderInserts (orders + order_items + order_status_history + order_offers)
```

Phase 15: Final Report

1. Executive Summary

Bait Al Mandi is a fully integrated restaurant order management system built on **Next.js 14** with **Supabase** as the database. The system provides a complete solution for managing a Yemeni restaurant including: digital menu, offers and bundles, delivery orders with real-time tracking, electronic invoices, and a comprehensive admin dashboard.

2. Architecture Overview

- **Framework:** Next.js 14 (App Router)
- **Language:** TypeScript
- **Styling:** TailwindCSS + CSS Variables (Dark/Light mode)
- **Database:** Supabase (PostgreSQL) + Drizzle ORM
- **State:** Zustand + localStorage persist
- **Auth:** Supabase Auth + RLS
- **Maps:** Leaflet + OSRM routing
- **PDF:** jsPDF + html2canvas
- **Analytics:** PostHog
- **Monitoring:** Sentry

3. Database Overview

- **22 tables** in the database (including Enums)
- **12 migration files**
- **Snapshot system** for historical data: order_items, order_offers, order_offer_items
- **RLS system**: Row-level security policies for public and admin tables

4. Technical Debt

1. **Duplicated calculateOfferPrice logic** between client and server
2. **Widespread use of any** instead of strict TypeScript
3. **No centralized Validation Layer**
4. **Supabase storage bucket misspelled**: "gellary" instead of "gallery"
5. **No automated tests** (Unit Tests)
6. **Some files mix Client and Server logic** without clear separation

5. Risks

Risk	Impact	Probability	Recommended Action
---	---	---	---
No Pagination in orders	Page slowness with growth	High	Add Pagination immediately
No Rate Limiting	Server flood potential	Medium	Add Rate Limiting
Duplicated offer calculation code	Price inconsistency	Low (resolved)	Periodic review
No tests	Difficulty detecting bugs	High	Add tests incrementally

6. Recommendations

1. **Immediate:** Add Pagination to orders page
2. **Immediate:** Add Indexes on sort fields (created_at, order_number)
3. **Short-term:** Create separate Service layer for business logic
4. **Short-term:** Add Rate Limiting for Server Actions
5. **Medium-term:** Write unit tests for core services
6. **Medium-term:** Refactor to Feature Modules
7. **Long-term:** Add CI/CD pipeline
8. **Long-term:** Improve LCP with Next/Image optimization

7. Future Roadmap

1. **Q1:** Add electronic payment via Yemeni payment gateways

2. **Q1:** Customer notification system (SMS/WhatsApp)
 3. **Q2:** Mobile application (React Native)
 4. **Q2:** Customer loyalty program and points system
 5. **Q3:** Integration with external delivery platforms
 6. **Q3:** Inventory management system
 7. **Q4:** Advanced reports with AI and predictive analytics
-

Document created on June 16, 2026

All rights reserved © 2026 Bait Al Mandi Restaurant



Thank You

BAIT AL MANDI — Order Management
System



DEVELOPED BY



Esnaad Tech

Technology Solutions & Software Development

June 16, 2026 | © 2026 All Rights Reserved